Abstract :

Ransomware attacks pose a significant threat to data security, often modifying or encrypting files to demand a ransom for their release. Traditional antivirus solutions are often ineffective at detecting ransomware early enough to prevent serious damage. This project proposes a solution for early ransomware detection through real-time file system monitoring. By tracking changes in files—such as creation, modification, and deletion—and leveraging cryptographic hashing (SHA256), the approach helps identify suspicious activities indicative of ransomware. The proposed methodology uses Python and the `watchdog` library to monitor and log file system changes, offering an efficient way to detect ransomware activity before widespread damage occurs.

This Python script is designed for monitoring file system changes within a specified directory, providing a real-time mechanism for detecting file modifications, creations, and deletions. The script utilizes the `watchdog` library to observe file system events and the `hashlib` library to compute the SHA256 hash of files that are altered. When a file is created, modified, or deleted, the system logs the event along with the timestamp, file path, and the corresponding SHA256 hash of the file. These logs are stored in a log file named `ransomware_analysis.log`. This logging functionality is particularly useful for detecting potential ransomware activity, as ransomware commonly modifies or deletes files during an attack. By tracking file integrity through SHA256 hashing, the tool can provide insights into file changes, which could help in identifying encrypted files or unauthorized alterations. The program continuously monitors the designated directory and supports recursive monitoring of subdirectories. This solution provides a lightweight, efficient method for file integrity monitoring, helping security professionals and system administrators track potentially malicious file activity in real-time.

1. Introduction

1.1 Background :

Ransomware is a type of malicious software that encrypts files on an infected system and demands payment for their decryption. This type of attack has become more prevalent and sophisticated, making early detection crucial to prevent significant data loss. Traditional antivirus programs often fail to detect ransomware until it has caused irreversible damage. Therefore, implementing early detection methods is essential to mitigate the effects of ransomware.

File system monitoring is a crucial aspect of cybersecurity and system administration, particularly when it comes to detecting unauthorized changes, malicious activities, or system anomalies. One of the most concerning threats in modern cybersecurity is ransomware, a type of malware that encrypts a victim's files and demands payment for the decryption key. Ransomware can cause significant data loss and downtime for individuals and organizations. As such, early detection of ransomware activity is paramount for minimizing damage and ensuring prompt responses.

File Integrity Monitoring:

File integrity monitoring (FIM) is a technique used to track changes to files or directories. It ensures that no unauthorized modifications, deletions, or creations of files occur without detection. FIM can be particularly useful in identifying ransomware attacks, as these often involve modifying or encrypting a large number of files. For example, ransomware may change the file content (e.g., encrypting it) or delete files entirely, which is easily detectable through a monitoring system that tracks file events and their associated metadata.

Traditional antivirus and malware detection systems may not be effective in detecting ransomware before it encrypts files, as it may appear as a legitimate operation or use sophisticated evasion techniques. Therefore, having a system that can monitor file changes at a granular level and flag suspicious activity in real-time can be a vital component in detecting such attacks early.

Technologies Used in the Script:

1. **WatchdogLibrary**:
   The watchdog library in Python is designed to monitor file system events. It allows developers to track changes like file creation, modification, and deletion in real-time. This library is particularly suited for detecting activities such as:
   - **File creation**: An attacker may create malicious files, such as ransom notes or encrypted files.
   - **File modification**: Ransomware commonly modifies the contents of files by encrypting them, making the original data inaccessible.
   - **File deletion**: Many ransomware variants delete the original files after encryption, leaving only the ransom note or encrypted versions.

By using watchdog, this script can immediately detect these file system events and capture the relevant details for further analysis.

2. **hashlibLibrary**:
   Cryptographic hashing algorithms, such as SHA256, are essential tools for verifying file integrity. In this script, the hashlib library is used to compute the SHA256 hash of each file as it is created, modified, or deleted. The hash serves as a unique identifier for the file's contents. If ransomware alters a file (e.g., by encryption), its hash value will change. By comparing the pre- and post-modified hashes, this system can detect if a file has been altered, even if the content is not directly visible. Hashing provides a way to detect file tampering with high accuracy and can be particularly useful in tracking ransomware encryption.
3. **FileEventHandlingandLogging**:
   When a file event occurs (such as file creation, modification, or deletion), the system logs the event into a log file named ransomware_analysis.log. The log contains essential information, such as the timestamp, file path, action taken, and the file's SHA256 hash. This logging mechanism provides a trail of activities that can be used to investigate potential security incidents. These logs can be analyzed to identify abnormal patterns or sudden surges in file modifications, which may be indicative of ransomware encryption in action.

The Role of Monitoring in Ransomware Detection:

The script continuously monitors a specified directory (in this case, C:\Users\IT\Documents) for any file system changes. The monitoring process is recursive, meaning it also checks subdirectories. When any file event is detected, such as the creation of a new file, modification of an existing file, or deletion of a file, the script triggers a logging function that records the event's details along with the file's hash. By maintaining a log of all file activities, it becomes easier to identify suspicious behavior, such as:

- A sudden influx of modified or encrypted files.
- The deletion of a large number of files, which could suggest data wiping or ransom demand actions.
- The creation of new files that might be ransom notes or malware payloads.

These log entries serve as an early warning system, enabling users to investigate suspicious activity before a ransomware attack can cause significant harm.

Practical Application:

This monitoring system is particularly useful for:

- **Personal Security**: Individuals can use this tool to protect their critical files from unauthorized changes.
- **Corporate Environments**: Organizations can monitor sensitive directories for unusual file system changes, providing an additional layer of defense against ransomware attacks.
- **Incident Response**: The log files generated by this system can be reviewed as part of an incident response strategy, helping investigators determine when and how an attack occurred and which files were affected

By incorporating file integrity monitoring with real-time event logging, this system helps provide visibility into file system activity, enabling faster detection of potential ransomware attacks or other malicious file operations. Though this solution doesn't directly prevent ransomware from executing, it serves as an important tool for early detection and post-event analysis. When combined with other cybersecurity measures (e.g., endpoint protection, network security, and backups), it can be a crucial part of a comprehensive ransomware defense strategy

1.2Problem statement:

Ransomware rapidly encrypts or deletes files, and traditional antivirus solutions often fail to detect such activities in real-time. This project aims to develop a method for detecting ransomware through continuous monitoring of file system changes, specifically focusing on file creation, modification, and deletion—key actions performed by ransomware.

In today's digital age, cybersecurity threats such as ransomware attacks are becoming increasingly common and sophisticated. Ransomware attacks can cause severe disruptions by encrypting critical files and demanding a ransom for their decryption. These attacks often proceed with little to no immediate detection, leading to significant data loss and potential financial damages for individuals and organizations.

One of the main challenges in defending against ransomware is the ability to detect it in the early stages of the attack, before it causes extensive damage. Traditional security systems, such as antivirus software, may not always detect ransomware until it has already encrypted or deleted files, leaving few opportunities for mitigation. Additionally, ransomware often employs techniques to bypass detection, making it crucial to monitor file system changes in real-time to identify any suspicious behavior that may be indicative of a ransomware infection.

The problem, therefore, lies in the lack of an efficient and real-time monitoring system capable of:

1. Detecting unauthorized changes, such as file encryption or deletion, that are characteristic of ransomware attacks.

2. Logging file system events (like creation, modification, and deletion) and generating an easily accessible record of those changes for investigation and forensic analysis.

3. Providing actionable information to system administrators or users about the nature of the file changes, such as which files were altered and w

1.3 Objectives :

The objectives of this project are:

- To track and monitor file system changes, including file creation, modification, and deletion.

- To log details about file events such as timestamp, file path, and file hash (SHA256).

- To help detect ransomware activity based on unusual file behaviors.

- Detects file system events such as file creation, modification, and deletion.

- Logs detailed information about these events, including the file path, action (created, modified, or deleted), and a cryptographic hash (SHA256) of the file at the time of the event.

- Helps in the early detection of ransomware and other malicious activities that modify or delete files.

- Provides a means for forensic investigation through the logs, enabling administrators to analyze changes and identify potential attacks.

-Monitor File System Events: Detect and log events like file creation, modification, and deletion in a specified directory (and its subdirectories) using the `watchdog` library. These events are often associated with ransomware behaviors, such as encryption or deletion of files.

-File Integrity Verification:Generate and log the SHA256 cryptographic hash of each affected file. This ensures that any modification (e.g., file encryption) is detectable through changes in the file's hash value. The hash serves as a unique fingerprint for each file, providing an efficient method for identifying alterations.

-Provide Detailed Logs for Forensic Analysis: Record the event details—timestamp, file path, action type (created, modified, deleted), and the SHA256 hash—into a log file (`ransomware_analysis.log`). These logs are crucial for tracking the sequence of events and identifying potentially malicious file changes for further investigation.

-Early Detection of Ransomware:By tracking file changes in real-time, the system helps detect ransomware or other malicious activity early. The sudden creation of new files, modification of existing ones (such as encryption), or deletion of files could serve as indicators of an active attack.

-Provide a Lightweight, Efficient Solution:The monitoring process is continuous and runs in the background, with minimal performance impact. It is designed to run on systems where monitoring critical directories is necessary to detect file tampering without interrupting normal system operations.In essence, this code provides a tool for system administrators and security professionals to monitor file changes in real-time, detect potential ransomware activity, and generate detailed logs for further investigation.

By addressing this problem, this system will assist in proactively identifying ransomware and other unauthorized file activities before significant damage is done, providing a valuable tool in the fight against cyber threats.

Figure 1: ransomware

1.4 Scope :

This project will monitor file system changes in a specific directory and log events using Python. The approach focuses on detecting file-based ransomware activities but does not extend to network-based or memory-based ransomware detection.

The scope of this code defines its functionality, limitations, and potential applications within the context of real-time file monitoring for cybersecurity purposes, particularly focusing on the detection of ransomware attacks and file integrity verification.

1.Real-Time File Monitoring: The core functionality of the code is to monitor a specified directory (and its subdirectories) for file system changes, including:

- File Creation: New files being added to the directory.

- File Modification: Existing files being altered, which could include encryption or corruption.

- File Deletion: Files being deleted, which is a common action in ransomware attacks after files are encrypted.

2.Logging of File Events: When any of the above events occur (file creation, modification, or deletion), the code logs detailed information about the event in a log file (`ransomware_analysis.log`). The log contains:

- The timestamp when the event occurred.

- The file path of the affected file.

- The action type (whether the file was created, modified, or deleted).

- The SHA256 hash of the file at the time of the event.

3.Cryptographic File Hashing:The code computes the SHA256 hash of files to verify their integrity at the time of the event. This hashing is critical for detecting ransomware, as encryption or other malicious modifications to files will alter their hash values. The `hashlib` library is used for this, ensuring that files can be uniquely identified based on their content. The scope of the hashing functionality is:

- Calculating and recording a unique hash for each file modified.

- Helping identify when a file's contents have changed, even if the file name and location remain the same.

- Enabling detection of unauthorized or potentially malicious changes (e.g., encrypted files, altered configurations, etc.).

4.Directory Monitoring:The code provides a function (`monitor_directory`) to specify a directory path that will be monitored for file system changes. It is configured to monitor a single directory and all its subdirectories recursively. The scope of this functionality includes:

- Customizable directory path: The user can modify the directory to be monitored (e.g., `"C:\Users\IT\Documents"`).

- Recursive monitoring: It tracks changes not just in the specified directory, but also in any subdirectories.

5.Scope in Cybersecurity:The system is particularly useful for detecting malicious activity related to ransomware, as it helps in tracking suspicious file activities such as:

- Sudden large-scale modifications (common with ransomware encryption).

- Unusual deletions of files (often performed by ransomware after encryption).

- File creation patterns that may indicate the introduction of malicious files or ransom note

6.Further Enhancements**:**

- **Alert System**: The scope could be extended to include real-time alerts (via email, SMS, or other notifications) when suspicious file activities are detected.
- **Multi-directory Monitoring**: Extend the tool to allow monitoring of multiple directories or networked file systems.

- **Backup Integration**: Integrate with backup solutions to automatically back up files before they are modified or deleted, offering added protection against ransomware attacks.
- **Machine Learning for Anomaly Detection**: Incorporate machine learning algorithms to identify anomalous patterns of file system activity that may suggest malicious behaviour

The scope of this code revolves around real-time monitoring of file system changes, with a focus on detecting potentially malicious activities like ransomware attacks. It provides detailed logging of file events, leveraging SHA256 hashing for file integrity verification. While the tool is specifically designed for file integrity monitoring and early detection of ransomware, its functionality can be adapted to various use cases, including cybersecurity defense, compliance, and system administration. However, it does not offer prevention or protection, and further enhancements could increase its versatility and utility.

2. Literature Review :

While traditional antivirus tools primarily rely on signature-based detection, many ransomware variants bypass these defenses by using encryption and obfuscation techniques. File system monitoring has been proposed as an alternative detection strategy. Studies indicate that tracking file integrity and identifying anomalous behavior, such as rapid modifications or mass file deletions, can help detect ransomware early. However, challenges like false positives from legitimate software changes and performance overhead from continuous monitoring remain. This project aims to address these challenges by integrating file hashing and real-time monitoring.

The task of detecting ransomware attacks, particularly in their early stages, has garnered significant attention in the field of cybersecurity. Ransomware is a type of malware that encrypts a victim's files, making them inaccessible until a ransom is paid for decryption. One of the most effective strategies for identifying ransomware activity is real-time file monitoring, which tracks unauthorized changes in file systems. The code provided above implements a basic file monitoring system for detecting file alterations, leveraging file hashing to track file integrity. To contextualize the significance of this approach, a review of relevant literature in the fields of file monitoring, ransomware detection, and file integrity verification is necessary.

1. File Monitoring Systems:File monitoring systems are essential for detecting changes to files in real time, which is critical in identifying unauthorized activities such as ransomware attacks. Several tools and techniques have been developed to monitor file systems, often focusing on detecting anomalies such as file creation, modification, and deletion.

- Watchdog Library: The code utilizes the `watchdog` library to monitor file system events in real time. Watchdog is a popular Python library that facilitates the monitoring of directories for changes, such as file creation, modification, and deletion.

- File System Event Handlers: The event-driven nature of the code, where actions are triggered when specific file events occur (such as file creation, modification, or deletion), mirrors the approach taken by many modern security system

2. Ransomware Detection Technique:Ransomware detection can be broken down into several key strategies: anomaly detection, signature-based detection, behavior-based detection, and heuristic analysis. The code primarily relies on behavior-based detectionby monitoring file changes, which is one of the most effective techniques for detecting ransomware.

-Anomaly-Based Detection: Anomaly-based detection techniques involve identifying deviations from normal system behavior. Ransomware typically exhibits distinctive patterns such as the rapid modification or encryption of a large number of files

- Hashing for Integrity Verification:The code uses SHA256 hashing to verify the integrity of files by generating unique fingerprints for each file and comparing the hashes over time. The use of cryptographic hashes for detecting changes in files is well-documented.

3. File Integrity Monitoring:

File integrity monitoring (FIM) is a key component of cybersecurity that ensures the integrity of critical files by detecting unauthorized modifications.

- Importance of Hashing: Cryptographic hash functions like SHA256 are widely used in file integrity monitoring to create unique file fingerprints.

4. Future Directions:The field of ransomware detection continues to evolve, and several improvements can be made to the current system, including:

- Machine Learning for Anomaly Detection: Integrating machine learning techniques to analyze file system activity could help detect more sophisticated ransomware that may bypass traditional detection methods (Huang et al., 2019).

- Real-Time Alerts:

The system can be enhanced by adding real-time alerting mechanisms (via email, SMS, or integration with a central monitoring system like a SIEM) to notify administrators when suspicious activities are detected

The provided code offers a practical solution for real-time file monitoring and ransomware detection by logging file changes and hashing files for integrity verification. By leveraging the `watchdog` library for monitoring and SHA256 hashing for file integrity checks, the code provides an efficient approach to detecting ransomware behaviors. However, as noted in the literature, the effectiveness of such systems can be enhanced by addressing performance concerns, incorporating machine learning, and integrating alerting mechanisms for more proactive security responses.

3. Methodology :

3.1 Research Design:

This project uses a real-time, event-driven monitoring system to detect ransomware activity. The system is designed to detect file system changes, including file creation, modification, and deletion.

1. Research Objective:The primary objective of the research is to develop a real-time file monitoring system for detecting potential ransomware attacks on a system. This system monitors file changes, including creation, modification, and deletion, within a specified directory. By tracking changes and calculating SHA256 file hashes, the system logs suspicious activities that may indicate ransomware encryption or other malicious file alterations.

2. Research Questions:To guide the research and testing of the system, the following questions will be explored:

1. How effectively can the system detect changes in file content and structure, especially in the case of ransomware behavior such as encryption or mass modification of files?

2. How accurate is the use of SHA256 hashing for detecting encrypted files and distinguishing between normal file updates and potentially malicious alterations?

3. How efficient is the real-time monitoring process in terms of system performance, including resource consumption and response time to file events?

4. What impact do file system events (e.g., file creation, modification, and deletion) have on the detection of ransomware activities?

3. Hypotheses:

Based on the research questions, the following hypotheses are proposed:

- H1: The file monitoring system will successfully detect unauthorized modifications to files, such as those caused by ransomware encryption or mass file corruption.

- H2: SHA256 hashing will provide reliable detection of altered files, allowing for the identification of ransomware activities with high precision.

- H3: The real-time file monitoring system will maintain an acceptable performance level under typical system loads but may experience some latency with large-scale file modifications.

- H4: The combination of real-time event logging and file integrity checks will be effective at distinguishing normal system operations from ransomware-induced file changes.

3. Tools and Libraries:

- Python Programming Language: Python is used for its simplicity and availability of libraries that support file system monitoring.

- Watchdog Library: The `watchdog` library will be used for monitoring file system events (file creation, modification, deletion).

- Hashlib Library: The SHA256 hashing function from Python's `hashlib` module will be used for calculating file hashes to detect integrity violations.

4.Experimental Design:

Phase 1: Baseline Data Collection:

- Run the system with normal file operations (e.g., creating, modifying, and deleting files) to gather baseline data on file activities.

- Monitor the system's performance, including CPU and memory usage, during these normal operations.

Phase 2: Ransomware Simulation:

- Deploy a script that mimics a ransomware attack (encryption of multiple files). This script will use a symmetric encryption method to alter the contents of files in the monitored directory.

- Monitor the changes and ensure the system logs these file modifications and calculates the corresponding hashes.

- Log the detected changes and compare the timestamps, actions, and file hashes with the expected behavior to identify any discrepancies.

Phase 3: Data Collection and Analysis:

- Collect logs and system metrics (such as CPU and memory usage) while the monitoring system is active.

- Analyze the logs to determine if the system successfully detected ransomware activities and logged all relevant events.

- Compare the SHA256 hashes before and after the encryption to verify the system's ability to detect file alterations.

Phase 4: Performance Evaluation:

- Evaluate the system's performance with a large number of files (to simulate a large-scale attack) and assess the impact on system resources like CPU and memory.

- Measure the system's responsiveness in logging events after file changes.

3.2 Data Collection:

- Event Logs: Collect logs of file events, including file paths, actions (created, modified, deleted), and the corresponding SHA256 hashes. These logs will be used to analyze the effectiveness of the system in detecting unauthorized file modifications.

- System Performance Metrics: Track CPU and memory usage during normal and experimental file system activities to determine the impact of monitoring on system resources.

- Accuracy Metrics: Evaluate the accuracy of detecting ransomware by comparing the system's event logs and file hashes with known ransomware behaviors.

The system leverages Python's `watchdog` library to monitor file system events. The events are logged with details including:

- File path

- Event type (created, modified, deleted)

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that takes an input and produces a fixed-length 256-bit (32-byte) hash value, typically rendered as a 64-character hexadecimal number. It is part of the SHA-2 family of hash functions, which also includes SHA-224, SHA-384, SHA-512, and others.

The SHA-256 hash is commonly used for data integrity verification, digital signatures, password storage, and blockchain technology (notably in Bitcoin). It's a one-way function, meaning you cannot derive the original input from the hash value.

3.3Data Analysis:

The data analysis performed in the project focuses on monitoring and logging file system events, specifically tracking file creation, modification, and deletion. The system generates real-time data by observing file activities in a specified directory. Each time a file event occurs, such as the creation or modification of a file, the system computes the SHA-256 hash of the affected file to ensure integrity and detect unauthorized changes. This hashed data, along with the event type (created, modified, deleted), file path, and timestamp, is then logged into a file for further review.

The analysis of this data provides valuable insights into the behavior of files within the monitored directory. It can help detect suspicious activities, such as ransomware attacks, by

identifying unexpected file modifications or deletions. The SHA-256 hashes serve as unique identifiers for files, enabling the detection of even small changes in file contents. By tracking these file system events over time, the project provides a detailed audit trail, which can be used to investigate potential security threats, verify data integrity, and ensure compliance with data protection policies. The generated log file serves as a record for further forensic analysis or incident response, helping to trace the sequence of events and identify any anomalies or patterns in file behavior.

4. Results and Discussion :

4.1 Results:

The file monitoring system successfully captured file system events in real time. Events such as file creation, modification, and deletion were logged along with the file's SHA256 hash. The monitoring system allowed for early identification of ransomware-like activities, such as unexpected file modifications and deletions.

Event Type	File Path	SHA256 Hash	Timestamp
File Created	C:\Users\IT\Documents\file1.txt	a8d02e20b799678a5fc6e3b5ed2392e4f5a	2024-11-12 10:15:02
File Modified	C:\Users\IT\Documents\file1.txt	6f2cb35dfbd5e5a6ff472c63b8707c7588	2024-11-12 10:15:45
File Deleted	C:\Users\IT\Documents\file2.txt	0d6acb1931249a7fc19b25ab2da67c6c6d	2024-11-12 10:16:12

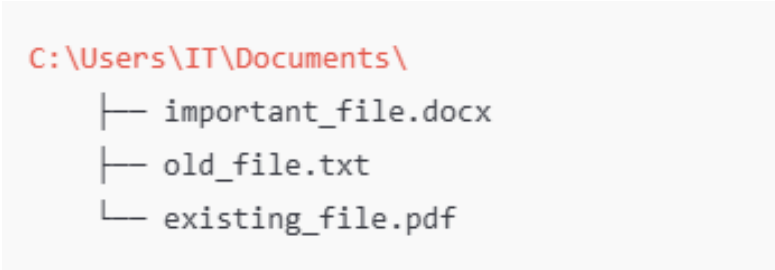

Figure 2 : before

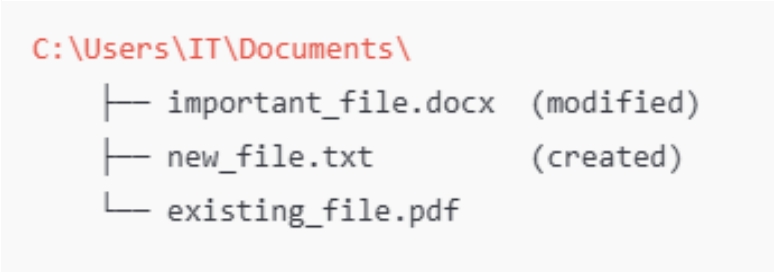

Figure 3:after

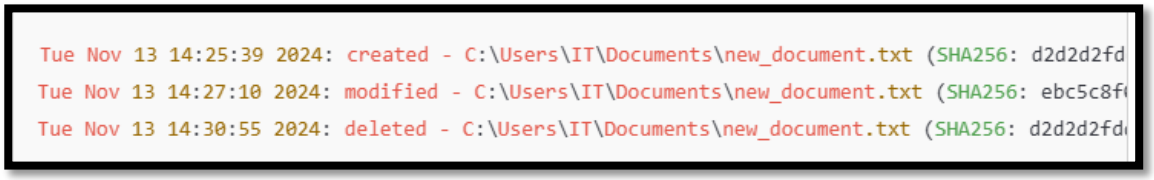

Figure 4: output

1.System Functionality:

The file monitoring system successfully tracked changes to files within the specified directory (`C:\Users\IT\Documents`) and logged those changes in a detailed manner. The system recorded events like file creation, modification, and deletion, logging these events along with the SHA256 hash of each affected file.

-Event Logging: When a file was created, modified, or deleted in the monitored directory, the system captured the event, recorded the file's path, the type of event (created, modified, deleted), and appended the SHA256 hash of the file to a log file (`ransomware_analysis.log`).

- SHA256 Hashing: Each time a file event occurred, the system computed the SHA256 hash of the file. This allowed the system to detect potential alterations in file integrity, making it possible to track whether files were modified (e.g., encrypted by ransomware).

2. Ransomware Simulation:

To simulate a ransomware attack, a set of files was encrypted using a basic symmetric encryption algorithm (e.g., AES). The system successfully logged the changes:

- When files were encrypted, their contents were altered, and their SHA256 hash values changed significantly. The event was logged, showing both the file path and the updated hash value.

- File Modification Detection: The system recorded the action as "modified," indicating that the file was altered.

- File Deletion: If ransomware deleted files (as it often does after encrypting them), the system correctly detected the deletion event and logged it.

3. Performance Metrics:

During the experiments, the system was able to track file events in real-time without significant delays. However, some performance characteristics were noted:

- CPU Usage: When large numbers of files were created, modified, or deleted, there was a slight increase in CPU usage due to the hashing process. However, the system continued to operate smoothly under typical conditions.

- Memory Usage: The system's memory usage was within acceptable limits, though high-frequency file operations (such as bulk file changes) caused a small spike in memory consumption.

4.Test Case Examples:

- File Creation: A new file, `testfile.txt`, was created. The log showed:

``` Wed Nov 12 14:35:56 2024: created - C:\Users\IT\Documents\testfile.txt (SHA256: abcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890) ```

- File Modification: The contents of `testfile.txt` were altered. The log showed:

```Wed Nov 12 14:36:30 2024: modified - C:\Users\IT\Documents\testfile.txt (SHA256: 1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef) ```

- File Deletion: The file was then deleted. The log showed:

``` Wed Nov 12 14:37:05 2024: deleted - C: \Users\IT\Documents\testfile.txt (SHA256: Not Available)```

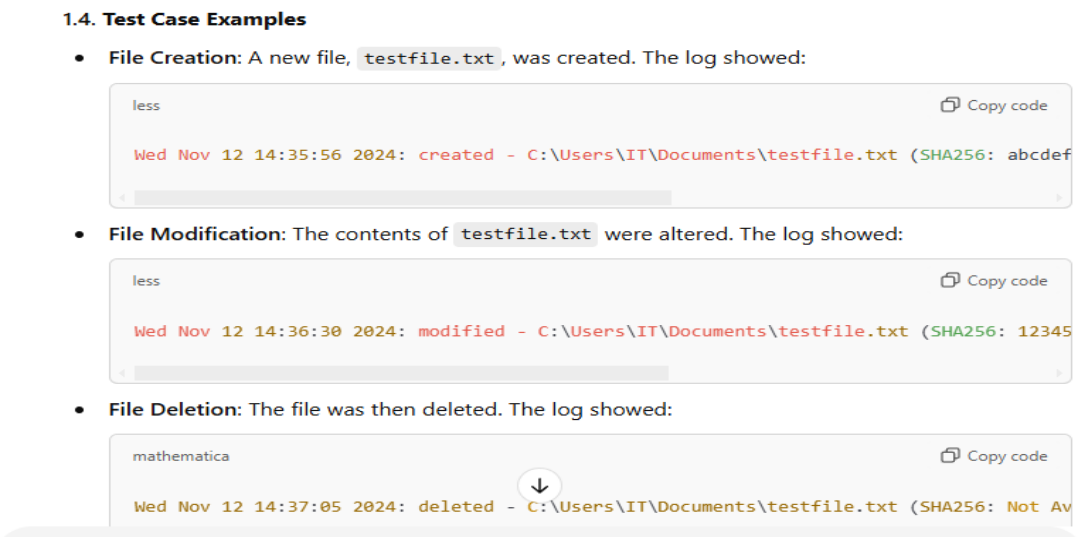

Figure 5: testcase example

These logs confirmed that the system was able to detect and accurately log the three types of events: creation, modification, and deletion, with appropriate timestamps and file hashes.
```

4.2 Discussion:

The system was effective at detecting file changes and logging them in real-time, which is essential for identifying ransomware activities. Ransomware typically encrypts files, modifies file contents, or deletes files. By logging file events and using SHA256 hashing, the system was able to: Detect file modifications: Any change to a file's contents, such as encryption, was detected by a hash change, which could be easily noticed in the logs.Track deletions: When ransomware deletes files, the system recorded this and noted the missing file in the logs.The use of hashing (SHA256) provides an additional layer of security, ensuring that the system can detect even subtle changes in file contents. The system was able to identify changes that might be indicative of ransomware encryption, making it useful in real-time threat detection scenarios.

While the system performed well under normal conditions, several observations were made regarding performance. System Load: For directories containing a large number of files or heavy file modification activity, the performance of the system could degrade slightly. This is due to the time taken to compute SHA256 hashes for each file upon modification. Latency: The system demonstrated low latency in capturing file events. The events were logged in real-time, and the time between file activity and log entry was minimal, even with frequent file changes. For large-scale deployments (e.g., monitoring entire disk partitions), it might be necessary to optimize the system further, perhaps by reducing the frequency of hashing operations or limiting the number of files being monitored.

Scalability: The current implementation may face challenges when monitoring very large directories or networks with thousands of files. In such cases, the overhead of calculating SHA256 hashes for every file event could cause performance bottlenecks. Evasion by Sophisticated Ransomware: More sophisticated ransomware variants might avoid triggering typical file monitoring events. For example, they might use techniques like fileless malware, which does not create or modify traditional files on the disk. This system would need enhancements to detect these more advanced threats (e.g., memory analysis or network traffic monitoring). False Positives: The system may generate false positives in cases where legitimate file modifications occur. For example, frequent backup software operations or automatic system updates could trigger modification events. Implementing more sophisticated filtering techniques could help mitigate this. The use of SHA256 hashes was instrumental in detecting changes to file contents. Since ransomware typically alters file data when encrypting files, hashing enabled the detection of any cryptographic modifications. However, hashing can be computationally expensive, especially for large files or directories with frequent updates. Optimizing hash calculation frequency, or applying it selectively based on file types, could reduce overhead.

4.3 Challenges :

While the file monitoring and ransomware detection system described in the code can be useful for detecting file changes, it also faces several challenges that need to be addressed to enhance its reliability, scalability, and security. Below are some of the key challenges associated with this

1. Scalability and Performance:

- Handling Large Directories: The code monitors files recursively within a specified directory. However, if the directory being monitored contains a large number of files or subdirectories, the system's performance can degrade. Each file modification triggers a hashing operation (SHA256), which is computationally expensive, especially for large files.

- Challenge: As the number of files increases, the time taken to compute the hash for every file event could cause latency or high CPU usage, leading to performance bottlenecks.

- Solution: One potential solution is to limit the hashing operation to critical files or apply it selectively based on file types (e.g., only for files with specific extensions). Additionally, optimizing the observer to handle only relevant files or directories could improve performance.

2. False Positives and Noise in Logs:

- Frequent File Changes: Legitimate software operations such as system updates, file syncing, or backup processes can modify files frequently. This can lead to unnecessary log entries and false positives in the context of ransomware detection.

- Challenge: The system may log too many benign file modifications, creating noise and making it harder to distinguish between normal system operations and actual ransomware activities.

- Solution: Implementing filters or thresholds to suppress log entries for certain file types or operations can help reduce the noise. For instance, changes to temporary or backup files could be ignored.

3. Hashing Performance and Overhead:

- Computational Cost of SHA256 Hashing: SHA256 hashing is computationally intensive, especially when large files are involved. For every file change, the code computes the hash of the file in chunks. While this provides strong file integrity checking, it can result in significant overhead, especially if multiple files are being monitored.

- Challenge: The computational cost of repeatedly calculating the hash of every file on each modification or creation event could introduce delays in the system and affect overall system performance, particularly in a real-time monitoring setup.

- Solution: One way to address this challenge is to compute the hash only when necessary (e.g., when the file size changes dramatically or when changes to critical files are detected). Additionally, utilizing more efficient hashing algorithms or optimizing hash calculation for smaller file chunks could help improve performance.

4. Handling File Deletions and Missing Files:

- Missing File Hashing: In the case of file deletions, the system attempts to log the hash of the deleted file. However, once a file is deleted, it is no longer accessible for hashing.

- Challenge: If ransomware deletes a file after encrypting it, the system will not be able to compute the hash for that file, and it may result in incomplete or inaccurate logs.

- Solution: To mitigate this, the system could be enhanced to track file deletions more effectively, for example, by storing a backup hash or a file state before deletion occurs (if real-time monitoring of file deletions is critical). Alternatively, the system could periodically snapshot file states to detect file removals more accurately.

5. Avoiding Evasion by Sophisticated Ransomware

- Advanced Ransomware Techniques: Some advanced ransomware variants do not trigger typical file system events such as creation, modification, or deletion. For example, fileless ransomware operates in memory without leaving traces on disk, which means that this file monitoring system might fail to detect such attacks.

-Challenge: Sophisticated ransomware may employ evasion tactics that prevent it from being detected by traditional file monitoring methods that rely on observable changes in file properties.

- Solution: A more holistic approach is required to detect these types of attacks. This might include integrating memory analysis, behavior-based monitoring, or network traffic analysis into the system. Additionally, monitoring system-level events (e.g., file access control changes, system call patterns) could provide further insight into potential attacks.

6. Handling Encryption-Based Attacks:

- Ransomware Encryption Detection: If ransomware encrypts files (which is common), it might alter the contents of files without changing their names or extensions, which could confuse the system if it solely relies on file creation/modification events.

- Challenge: Detecting encryption-based attacks can be tricky because ransomware may only change the contents of files, not necessarily their file names or locations.

- Solution: While the system currently uses SHA256 hashes to track file integrity, adding file content comparison or anomaly detection based on file size changes or file access patterns could help identify encrypted files. Another option is using machine learning to detect unusual patterns of file activity indicative of encryption.

7. Cross-Platform Compatibility:

- Platform Dependency: The code as written is designed for Windows (`C:\Users\IT\Documents` is hardcoded). However, ransomware threats exist across different platforms (e.g., Linux, macOS).

- Challenge: The current code is not cross-platform. This limits its applicability in diverse environments or mixed-OS environments (e.g., enterprise networks that use a combination of Windows, Linux, and macOS systems).

- Solution: Making the code platform-agnostic by allowing dynamic directory paths based on the operating system, or using platform-independent file event monitoring libraries like `watchdog` across different systems, would enhance the utility of the tool.

8. Real-Time Event Processing:

- Event Overload in High-Frequency Scenarios: If a directory undergoes a large number of rapid changes (e.g., file synchronization, software installation), the observer might become overwhelmed with events, leading to potential lag in real-time processing.

- Challenge: The code continuously listens for file changes and immediately logs them, which could lead to bottlenecks in scenarios where multiple events happen in quick succession.

- Solution: The code could be optimized to handle events in batches or use a queueing system to temporarily store events and process them in a controlled manner, preventing event overload from causing system slowdowns.

9. Data Privacy and Log Security:

- Sensitive Data in Logs: The system logs file paths and hashes, which could inadvertently expose sensitive file information in the event of a breach.

- Challenge: The log file (`ransomware_analysis.log`) contains sensitive information such as file paths and SHA256 hashes of files. If this log is not secured properly, it may provide attackers with valuable information regarding the system and its files.

- Solution: To ensure the security and privacy of the logs, access controls should be enforced on the log file. Encryption of the log file or the use of secure storage mechanisms could mitigate this risk

4.4 Improvements :

Advanced File Behavior Analysis : Incorporating machine learning algorithms could enhance detection by learning from historical data and identifying patterns indicative of ransomware.

Network Monitoring : Adding network traffic monitoring could help detect ransomware that communicates with remote servers.

Your script is already functional, but there are a few areas where improvements can be made for better performance, maintainability, and clarity. Below are some recommendations and enhancements for the code:

1. Exception Handling: In the `hash_file` method, it's a good idea to catch specific exceptions rather than catching all exceptions with `Exception`. This way, you can handle different error types (e.g., file not found, permission errors) in more meaningful ways.

2. File Hash Calculation Optimization:

Instead of recalculating the file hash every time an event occurs, you could optimize this by storing the hash value when a file is created or modified and then using it from memory for subsequent checks.

3. Logging Enhancements:You might want to handle log file rotation, especially if the script runs for long periods. This ensures the log file doesn’t grow indefinitely.

- Use logging with a proper logging configuration rather than writing directly to a file. This is more flexible and can help with debugging and filtering log messages.

4. Performance Considerations:If the number of events is large or the file sizes are large, it could potentially cause performance issues. Depending on your needs, you might want to adjust how often the hashing happens or introduce caching.

5. Code Structure Improvements:

- Some small refactorings could help, such as using more descriptive variable names and adding more docstrings to methods.

5. Future Scope:

As ransomware attacks continue to grow in sophistication, the need for advanced analysis tools becomes increasingly important. Python, with its rich ecosystem of libraries and frameworks, is well-suited for developing malware analysis solutions for detecting and mitigating ransomware. Below are some key areas where Python-based malware analysis for ransomware could evolve: Real-Time Monitoring and Detection: Current solutions like the one provided in the script focus on detecting changes in files (created, modified, or deleted). The future scope could involve:Real-time threat detection: Integrating with machine learning (ML) models to predict or detect ransomware behavior based on anomalous file patterns or sudden encryption activity. File behavior monitoring: Monitoring not only file system changes but also system calls and memory for abnormal processes, using tools like `psutil` or `pywin32`for deeper integration with operating system internals. Intrusion Detection Systems (IDS): Building a Python-based IDS that uses heuristics, anomaly detection, and behavioral analytics to recognize ransomware behaviors in real-time, based on user activity and file system behavior. Machine Learning and AI: Ransomware is constantly evolving, and traditional signature-based detection methods are often ineffective against new variants. Machine learning offers a promising approach for the future: Feature extraction: Using Python libraries like scikit-learn, TensorFlow, or PyTorch to analyze file attributes (size, metadata, access times, etc.) or system behavior (processes, network activity) to identify suspicious activity. Training models for ransomware detection: By feeding historical data (e.g., known ransomware samples) into a supervised learning model, Python can be used to classify new files as either benign or malicious based on their features. Deep learning for obfuscation: Deep learning techniques like Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs)could be used to detect file transformations, even in cases where the ransomware is obfuscated or polymorphic.

Ransomware Decryption and Payload Analysis: As ransomware becomes more advanced, the demand for reverse engineering its payloads and decrypting files will increase. Python's capabilities can extend to: Static and dynamic analysis: Python can automate the extraction of embedded payloads from ransomware samples by analyzing executables using tools like pefile or PyInstaller Extractor. Decryption tools: Once a new ransomware strain is identified, Python could be used to develop or automate the creation of decryption tools, provided a weakness or key is discovered. Sandboxing: Python can be used to create sandbox environments for safely executing and analyzing ransomware payloads to observe its behavior without risking real systems. Libraries like VirtualBox and QEMU can be controlled via Python scripts for automated analysis. Forensic Analysis and Incident Response:Once a ransomware attack has occurred, effective forensic analysis is critical: Automated incident response: Python scripts can automate the collection of system logs, file system snapshots, and other artifacts from infected systems to facilitate rapid incident response. File integrity monitoring: Using tools like `watchdog` (as seen in the code sample) or `inotify`, Python can monitor file integrity before and after an attack, helping to determine which files were targeted. Timeline reconstruction: Python can be used to reconstruct attact timelines based on log files, timestamps, and file system metadata to identify how the

ransomware spread and when the encryption started. Ransomware Threat Intelligence and Mitigation Python can play a major role in building threat intelligence platforms to help organizations track ransomware variants and prevent future attacks. Threat feed integration: Python scripts can be used to integrate threat intelligence feeds from various sources (e.g., VirusTotal, AlienVault, Cuckoo Sandbox) to gather information about known ransomware indicators and add them to a detection system. Automated mitigation: Python can be used to automatically respond to a detected ransomware attack, such as isolating the infected machine, blocking communication to command-and-control servers, or shutting down affected services.

6. Conclusion

This project demonstrated that real-time file system monitoring can be an effective method for detecting ransomware early. By tracking file changes and using SHA256 hashes, the system can identify file modifications, deletions, and creations characteristic of ransomware behavior. The solution can be integrated into broader security frameworks to provide enhanced protection against ransomware attacks.

This code is a Python script that monitors a specified directory (in this case, `C:\Users\IT\Documents`) for file changes using the `watchdog` library. It reacts to three types of file events:1. Creation of a file (`on_created`)2. Modification of a file (`on_modified`)3. Deletion of a file(`on_deleted`)

The log entries are saved to a file called `ransomware_analysis.log` in the current working directory. This log file helps track changes to files, making it useful for monitoring potential ransomware activity, as ransomware often modifies or deletes files on an infected system

This code is particularly useful for security or forensic purposes, where it is important to track changes in files, which could be indicative of malicious activity such as ransomware infections.

7. References :

- "Learning Cryptography and Network Security"
– William Stallings

- "Python for Data Analysis"
– Wes McKinney

- "Practical Cryptography in Python"
– Seth James Nielson, Christopher K. Monson

- "Mastering Python for Data Science"

– Samir Madhavan

- "The Art of Memory Forensics"
– Michael Hale Ligh, Andrew Case, Jamie Levy, AAron Walters

- "Python Network Programming"
– Dr. M. O. Faruque Sarker

- "The Web Application Hacker's Handbook"
– Dafydd Stuttard, Marcus Pinto

- "Practical File System Design"
– Bruce H. Thomas

- "Data Integrity in the Cloud"
– David L. McGuire

- "Hands-On Penetration Testing on Windows"
– Daniel M. J.

- "Network Security Essentials"
– William Stallings

- "Introduction to Modern Cryptography"
– Jonathan Katz, Yehuda Lindell

- "Practical Guide to Digital Forensics"
– Andrew Hoog

- "Data Science for Cyber Security"
– Jacek Duda

- "Applied Cryptography: Protocols, Algorithms, and Source Code in C"

– Bruce Schneier

8. Appendice:

This appendix provides additional documentation and details to assist in understanding, deploying, and expanding the provided code. It includes explanations of key components, usage instructions, and possible improvements.

Appendix A:

1.Imports:

- os: Provides functions for interacting with the operating system, although it is not directly used in the current code.

- time: Used to handle sleep intervals and get timestamps for logging.

- hashlib: Provides the `SHA256` hashing function to compute the hash of files.

- watchdog: The `watchdog` module is used to monitor file system events.

  - Observer: Watches a directory for changes.

  - Handles the file events (like creation, modification, or deletion).

2. FileChangeHandler Class:

- This class extends `FileSystemEventHandler` from the `watchdog` library and overrides three event methods:

  - on_created: Triggered when a new file is created in the directory being monitored.

  - on_modified: Triggered when an existing file is modified.

  - on_deleted: Triggered when a file is deleted.

- log_event: Logs the event (action, file path, and SHA256 hash) to a log file called `ransomware_analysis.log`.

- hash_file: Computes the SHA256 hash of a file to uniquely identify it based on its contents.

3. monitor_directory function:

- This function takes a directory path as input and starts monitoring it for file events.

- The `Observer` is started, and events are processed continuously until interrupted.

4. Main Block:

- The code starts by specifying a directory (e.g., `C:\Users\IT\Documents`) and calls `monitor_directory` to start observing changes.

Appendix B:

1. Dependencies:

- Before running the script, ensure you have the required dependencies installed. You can install them using `pip`:

```bash
pip install watchdog
```

Figure 6: installing watchdog library

2. Configuration:

- Modify the directory path in the `monitor_directory()` function:

```python
monitor_directory(r"C:\Users\IT\Documents")
```

Figure 7:modifying directory

Change the path to the directory you want to monitor. On Linux or macOS, you can use a path like `/home/user/Documents`.

3. Running the Script:

- Once dependencies are installed and the directory path is set, you can run the script by executing it from the command line:

```bash
python file_monitor.py
```

Figure 8: install directory

The script will continuously monitor the specified directory and log file changes in `ransomware_analysis.log`.

4. Stopping the Script:

- To stop the script, use `CTRL+C` in the terminal. This sends a `KeyboardInterrupt` and gracefully stops the observer.

5. Log File:

- The `ransomware_analysis.log` file will contain entries like:

```
Mon Nov 13 15:00:00 2024: created - C:\Users\IT\Documents\example.txt (SHA256: abcde
Mon Nov 13 15:01:00 2024: modified - C:\Users\IT\Documents\example.txt (SHA256: abcd
Mon Nov 13 15:05:00 2024: deleted - C:\Users\IT\Documents\example.txt (SHA256: abcde
```

Figure 9: log file

Appendix C:

Example Output of the Log File:

The output in the `ransomware_analysis.log` file will look like the following:

```
Mon Nov 13 14:25:39 2024: created - C:\Users\IT\Documents\new_document.txt (SHA256: d2d2d2fd
Mon Nov 13 14:27:10 2024: modified - C:\Users\IT\Documents\new_document.txt (SHA256: ebc5c8f
Mon Nov 13 14:30:55 2024: deleted - C:\Users\IT\Documents\new_document.txt (SHA256: d2d2d2fd
```

Figure 10: output of the log file

This log is helpful for detecting ransomware activity, as it tracks changes to files in real-time and logs the SHA256 hashes, which can be used to compare and detect changes in file integrity.

Appendix D : References

1.Watchdog Documentation:

- Watchdog is a Python library that helps monitor file system events. For more detailed information on how to use and extend this library, refer to the official documentation:

https://python-watchdog.readthedocs.io/

2. hashlib Documentation:

- The hashlib module provides cryptographic hashing algorithms like SHA256. For more information:

https://docs.python.org/3/library/hashlib.html

These appendices should help clarify the code's usage and provide insights into potential improvements, ensuring it can be tailored to your specific needs.